

Contents

1	Introduction	3
2	System Overview	4
2.1	Complete Planning Stack	4
3	Layer 1 : Raceline Generation	5
3.1	Objective	5
3.2	Why Splines?	5
3.3	Catmull-Rom Spline	6
3.4	Why Catmull-Rom?	6
4	Layer 2 : Curvature Based Speed Planner	7
4.1	Objective	7
4.2	Curvature	7
4.3	Maximum Speed	7
5	Layer 3 : Frenet Local Planner	9
5.1	Objective	9
5.2	Frenet Coordinates	9
5.3	Projection	9
5.4	Lookahead Target	10
6	Layer 4 : Minimum Jerk Trajectory	11
6.1	Objective	11
6.2	Jerk	11
6.3	Optimization Problem	11
6.4	Polynomial Solution	12
6.5	Minimum Jerk vs Minimum Snap	12
7	Layer 5 : Linear MPC	13
7.1	Objective	13
7.2	State Vector	13
7.3	Control Vector	13
7.4	System Dynamics	14
7.5	Cost Function	14
7.6	Constraints	14
8	Implementation Plan	15
8.1	Phase 1	15
8.2	Phase 2	15
8.3	Phase 3	15
8.4	Phase 4	15

8.5	Future Upgrades	16
-----	---------------------------	----

Chapter 1

Introduction

This document presents the proposed planning architecture for Team Singularity's autonomous drone racing system.

The perception stack is assumed to provide:

- Gate positions
- Gate orientations
- Drone state estimate

The objective of the planning stack is to convert these inputs into low level control commands.

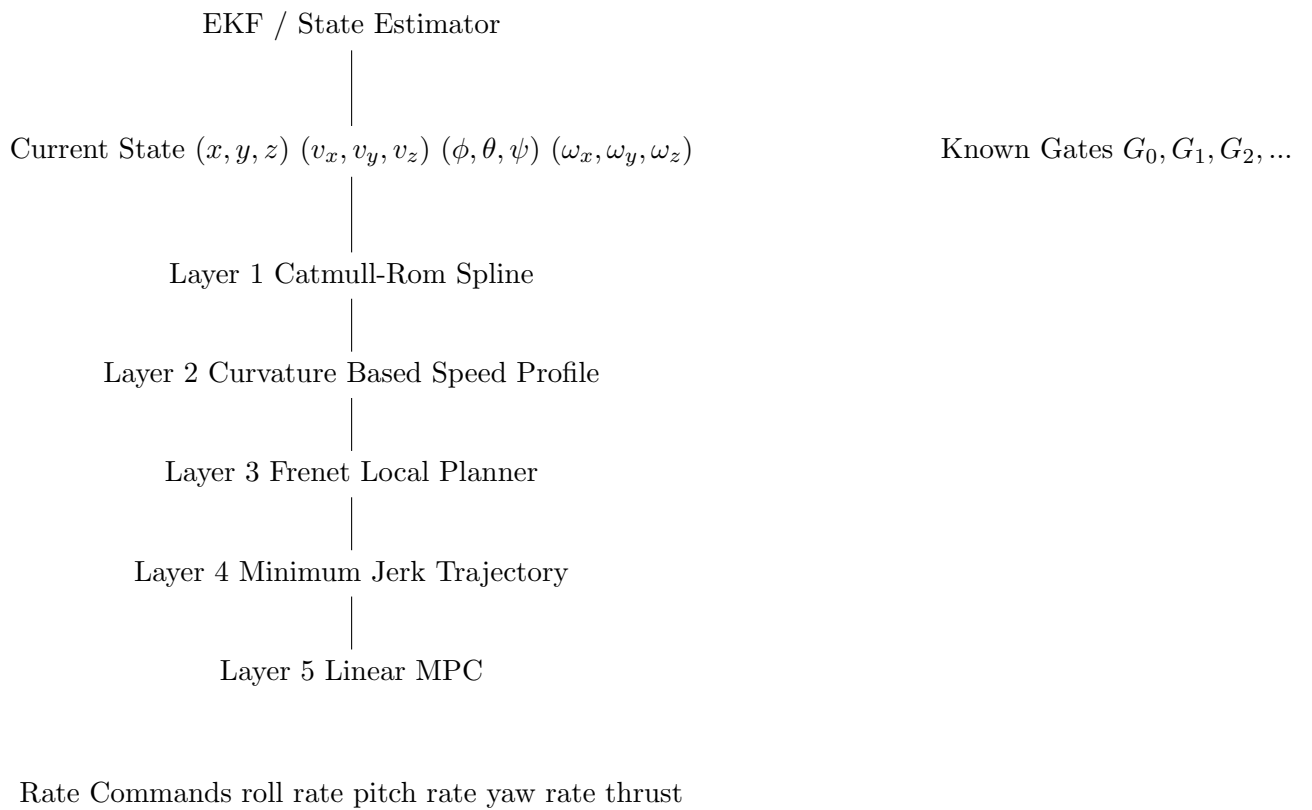
The proposed architecture consists of five layers:

1. Raceline Generator
2. Speed Planner
3. Frenet Local Planner
4. Minimum Jerk Trajectory Generator
5. Linear Model Predictive Controller

Chapter 2

System Overview

2.1 Complete Planning Stack



Chapter 3

Layer 1 : Raceline Generation

3.1 Objective

The objective of Layer 1 is to determine a smooth path through the racing gates.
Given

$$G_i = (x_i, y_i, z_i)$$

where

- x_i : x coordinate of gate i
- y_i : y coordinate of gate i
- z_i : z coordinate of gate i

the output is

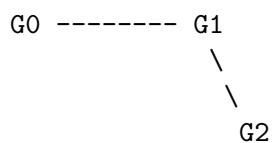
$$r(s) = [x(s), y(s), z(s)]$$

where

- s : arc length along the raceline
- $x(s)$: x coordinate
- $y(s)$: y coordinate
- $z(s)$: z coordinate

3.2 Why Splines?

Straight line segments create discontinuous headings.



This causes

- Infinite curvature
- Abrupt acceleration
- Aggressive roll commands

Instead, a spline provides

- Smooth position
- Smooth velocity
- Smooth acceleration

3.3 Catmull-Rom Spline

A Catmull-Rom spline passes exactly through the control points.

G0 ○

○ G1

○ G2

○ G3

The spline equation is

$$P(t) = \frac{1}{2} \left[2P_1 + (-P_0 + P_2)t + (2P_0 - 5P_1 + 4P_2 - P_3)t^2 + (-P_0 + 3P_1 - 3P_2 + P_3)t^3 \right]$$

where

- P_0, P_1, P_2, P_3 : neighboring control points
- $t \in [0, 1]$: interpolation parameter
- $P(t)$: point on spline

3.4 Why Catmull-Rom?

Advantages:

- Passes exactly through gates.
- Computationally inexpensive.
- Smooth first derivative.
- Easy future upgrade to optimized racelines.

Chapter 4

Layer 2 : Curvature Based Speed Planner

4.1 Objective

Determine the speed profile along the raceline.

Input:

$$r(s)$$

Output:

$$v(s)$$

4.2 Curvature

The curvature is

$$\kappa(s) = \frac{|r'(s) \times r''(s)|}{|r'(s)|^3}$$

where

- $\kappa(s)$: curvature
- $r'(s)$: tangent vector
- $r''(s)$: second derivative

4.3 Maximum Speed

The lateral acceleration is

$$a_{lat} = v^2 \kappa$$

where

- a_{lat} : lateral acceleration
- v : velocity
- κ : curvature

Rearranging,

$$v_{max} = \sqrt{\frac{a_{lat,max}}{\kappa}}$$

where

- v_{max} : maximum speed
- $a_{lat,max}$: maximum allowable lateral acceleration

Chapter 5

Layer 3 : Frenet Local Planner

5.1 Objective

The Frenet planner determines the local target point.

5.2 Frenet Coordinates

Instead of

$$(x, y, z)$$

the drone is represented by

$$(s, d, h)$$

where

- s : progress along track
- d : lateral deviation
- h : vertical deviation

5.3 Projection

Drone position

$$P_d = (x_d, y_d, z_d)$$

Nearest point on raceline

$$P_r = r(s)$$

Lateral error

$$d = ||P_d - P_r||$$

5.4 Lookahead Target

Raceline

X

*

Drone

The planner

1. Finds nearest point.
2. Moves forward by

$L_{lookahead}$

3. Selects

(x_t, y_t, z_t, v_t)

as target.

Chapter 6

Layer 4 : Minimum Jerk Trajectory

6.1 Objective

Generate a smooth trajectory from the current state to the local target.

6.2 Jerk

Jerk is defined as

$$J = \frac{d^3x}{dt^3}$$

where

- J : jerk
- $x(t)$: position
- t : time

Jerk is the rate of change of acceleration.

6.3 Optimization Problem

The objective is

$$\min \int_0^T \left(\frac{d^3x}{dt^3} \right)^2 dt$$

subject to

$$x(0) = x_0$$

$$x(T) = x_f$$

$$\dot{x}(0) = v_0$$

$$\dot{x}(T) = v_f$$

$$\ddot{x}(0) = a_0$$

$$\ddot{x}(T) = a_f$$

6.4 Polynomial Solution

The solution is

$$x(t) = a_0 + a_1t + a_2t^2 + a_3t^3 + a_4t^4 + a_5t^5$$

where

- a_i : polynomial coefficients
- t : time
- $x(t)$: position

6.5 Minimum Jerk vs Minimum Snap

Property	Minimum Jerk	Minimum Snap
Objective	d^3x/dt^3	d^4x/dt^4
Polynomial Order	5	7+
Complexity	Low	Medium
Smoothness	Excellent	Excellent

Chapter 7

Layer 5 : Linear MPC

7.1 Objective

Track the reference trajectory while satisfying drone constraints.

7.2 State Vector

$$x = [xyzv_xv_yv_z\phi\theta\psi\omega_x\omega_y\omega_z]$$

where

- x, y, z : position
- v_x, v_y, v_z : velocity
- ϕ : roll
- θ : pitch
- ψ : yaw
- $\omega_x, \omega_y, \omega_z$: angular velocity

7.3 Control Vector

$$u = [pqrT]$$

where

- p : roll rate
- q : pitch rate
- r : yaw rate
- T : thrust

7.4 System Dynamics

$$x_{k+1} = Ax_k + Bu_k$$

where

- A : state transition matrix
- B : control matrix
- x_k : current state
- u_k : control input

7.5 Cost Function

The MPC minimizes

$$J = \sum_{k=0}^N (x_k - x_{ref,k})^T Q (x_k - x_{ref,k}) + u_k^T R u_k$$

where

- N : prediction horizon
- Q : state cost matrix
- R : control cost matrix
- x_{ref} : reference trajectory

7.6 Constraints

$$|\phi| \leq \phi_{max}$$

$$|\theta| \leq \theta_{max}$$

$$T_{min} \leq T \leq T_{max}$$

8.5 Future Upgrades

